



**NANYANG
TECHNOLOGICAL
UNIVERSITY**

CE4055

LAB:3

Understanding Smart Cards Communication

AGENDA:

- ❑ Hands-on session with Smart Cards and understanding how to communicate with Smart Cards.
- ❑ We will introduce the students to the hardware and software components in a basic smart card setup and trying to familiarize with them.
- ❑ We will work with two types of Smart cards:
 - ❑ CEPAS 2.0 card (Your ID card/EZLink Card)
 - ❑ MiFARE Ultralight Memory Card

What is a Smart Card?



- ❑ A smart card is a credit card sized plastic card with an embedded computer chip.
- ❑ The chip can either be a microprocessor with internal memory or a memory chip with non-programmable logic.
- ❑ They can be programmed to accept, store and send data.

Difference between a Smart Card and a Magnetic Stripe card

☐ Magnetic Stripe card

- ☐ Does not have an embedded chip and mostly can only store data.
- ☐ No security of data
- ☐ Only Stores **meta data** of the card
- ☐ E.g. North Spine Card
- ☐ <https://www.youtube.com/watch?v=Wbh9-gkSdrU>
- ☐ <https://www.youtube.com/watch?v=IITY2XukZek>

☐ Smart card

- ☐ Can accommodate more information compared to magnetic stripe card.
- ☐ Has more powerful processing capabilities (can be used to perform operations such as data encryption, etc.)

Smart Card Readers



Computer based : needs to connect through USB or COM (Serial) ports

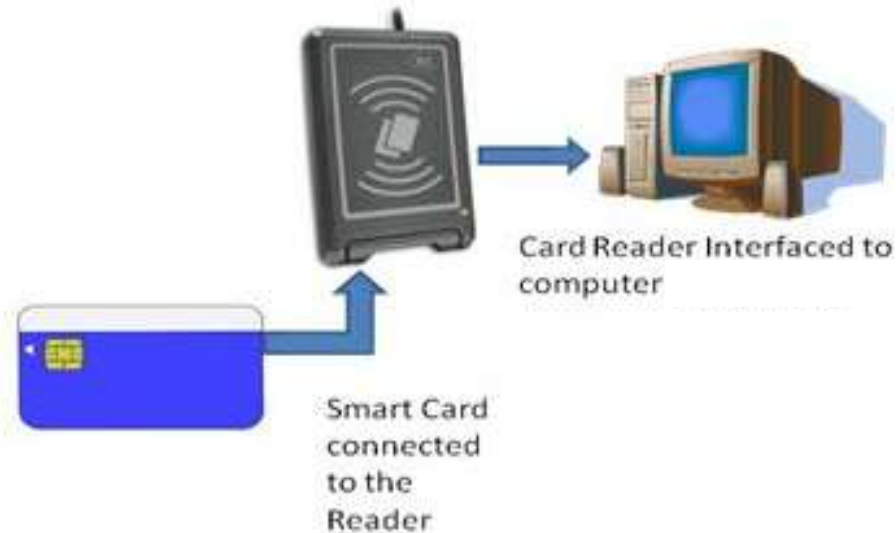
Dedicated Terminals: usually with a small screen, keyboard, etc.



Terminal/PC Card Interaction

- ☐ The terminal/PC sends commands to the card (through the serial line/USB).
- ☐ The card executes the command and sends back the reply.
- ☐ Sometimes, security measures are incorporated in the card wherein authentication is required you can start communicating with the smart card.
- ☐ In this Lab session, we are going to be dealing with **security-free** access.

A Basic Smart Card Working System



A Basic Smart Card Working System

- ❑ There are a number of communication interfaces here:
 - ❑ Smart Card to Smart card reader.
 - ❑ Smart Card Reader to Host-PC driver.
 - ❑ Host-PC software driver to User Application.

A Basic Smart Card Working System

- ❑ Back in the days, all the components involved in this setup were all **proprietary**.
- ❑ To understand its implications, consider the same scenario in the context of mobile phones.
 - ❑ Samsung phones could only talk with Samsung Cell phone towers and Samsung cell phones.
 - ❑ Samsung will have its own OS.
 - ❑ Samsung will have to develop its own Whatsapp Application or Temple run Application.
- ❑ It would be a real pain for all the Mobile phone companies and also its customers right?
- ❑ This was the case with Smart cards back in the days.

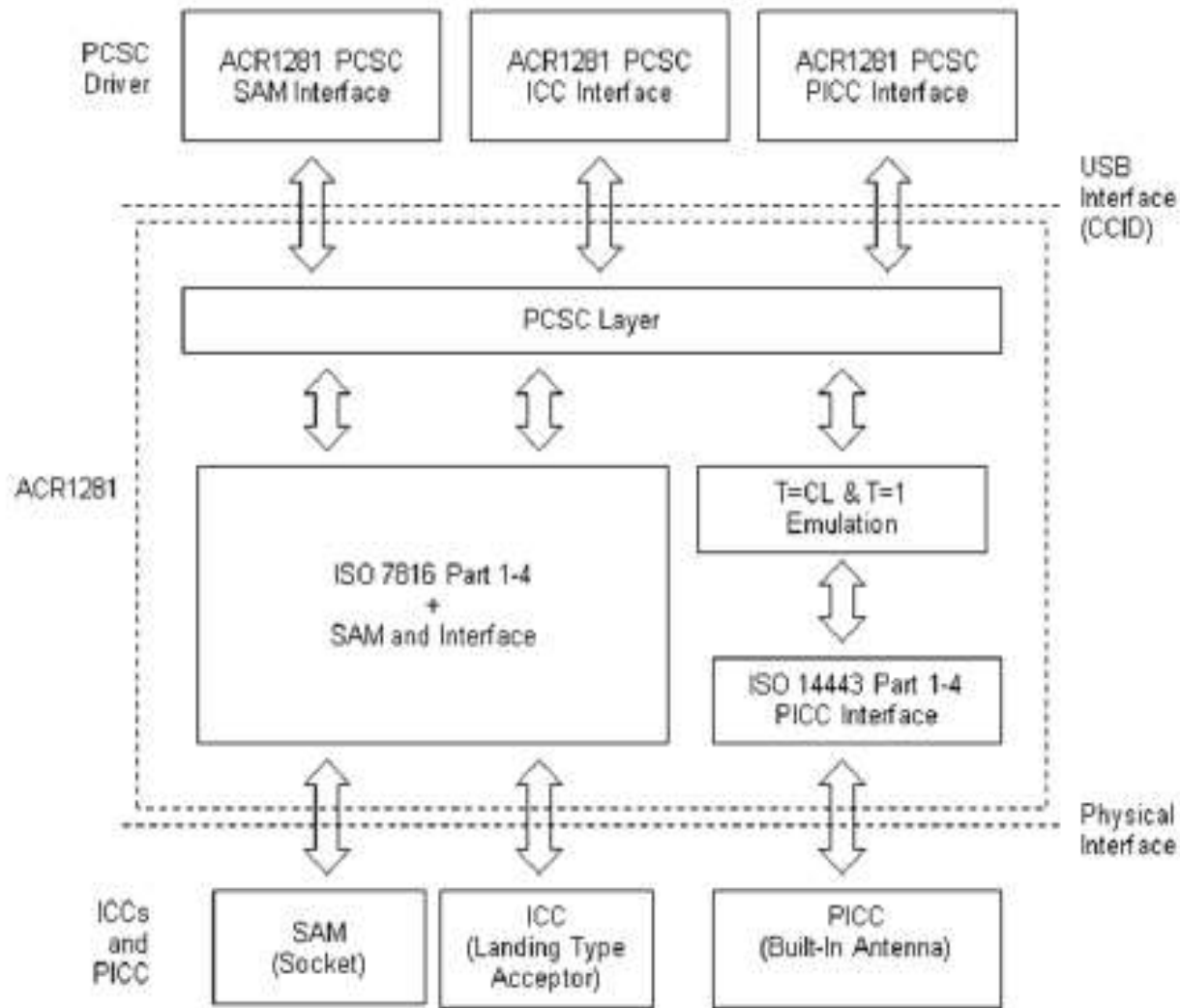
Standardizing everything!!!

- ❑ All the interfaces and communication between the same were standardized through a huge effort from big players like Microsoft, Motorola, NXP and others.
- ❑ Today, a lot of standards exist for each of the interfaces present in a typical smart card environment.
- ❑ ISO/IEC 7816, 14443 standards for the interface between the smart card and smart card reader.
- ❑ PC/SC standard for the interface between the smart card reader and Host-PC.
- ❑ CCID protocol to connect the smart card reader to host through USB.
- ❑ Chip operating systems for hosting multiple applications on a single smart card.

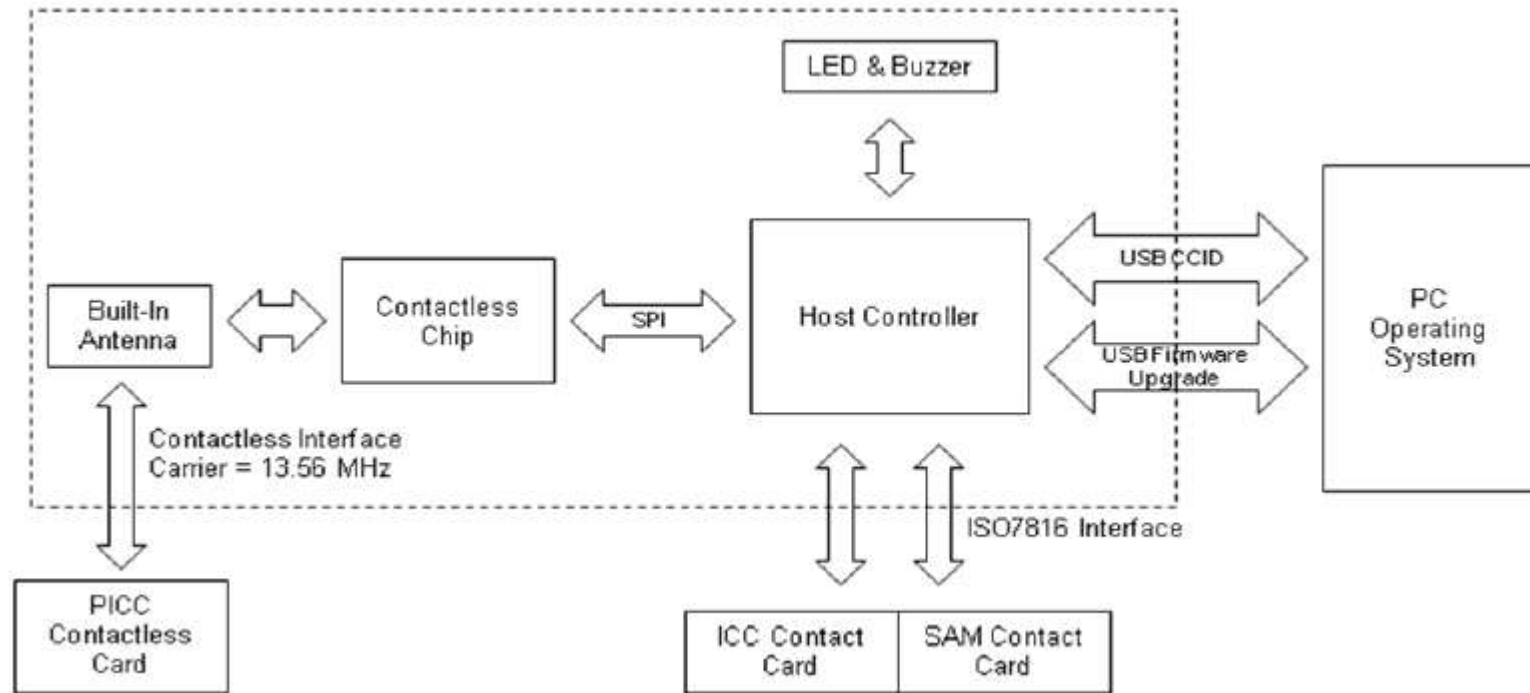
PC/SC Standards:

- ❑ Defines API for communication between personal computers and smart cards by the PC/SC working group.
- ❑ The standard provides tools for managing multiple smart card readers connected to the PC, involving both communicating with the reader and also the smart card.
- ❑ PC/SC standard has been implemented on Windows and is also implemented for Linux as PC/SC Lite.
- ❑ Any PC/SC compliant application can work with any PC/SC compliant smart card reader.
- ❑ Using these APIs, the application can detect readers, select them, configure them and communicate with the connected smart cards.

Communication between PC/SC driver and the smart card reader



Architecture of Smart Card Reader



Fun Facts:

☐ Full form for **NETS**?

☐ **Network for Electronic Transfers**

☐ Who owns **NETS**?

☐ **DBS Bank, OCBC Bank, UOB Bank.**

☐ When was **NETS** started?

☐ **1985!!!**

☐ What are the services that **NETS** provides?

☐ **NETS, eNETS, NETSPay, CashCard, FlashPay, FAST, PayNow, GIRO**

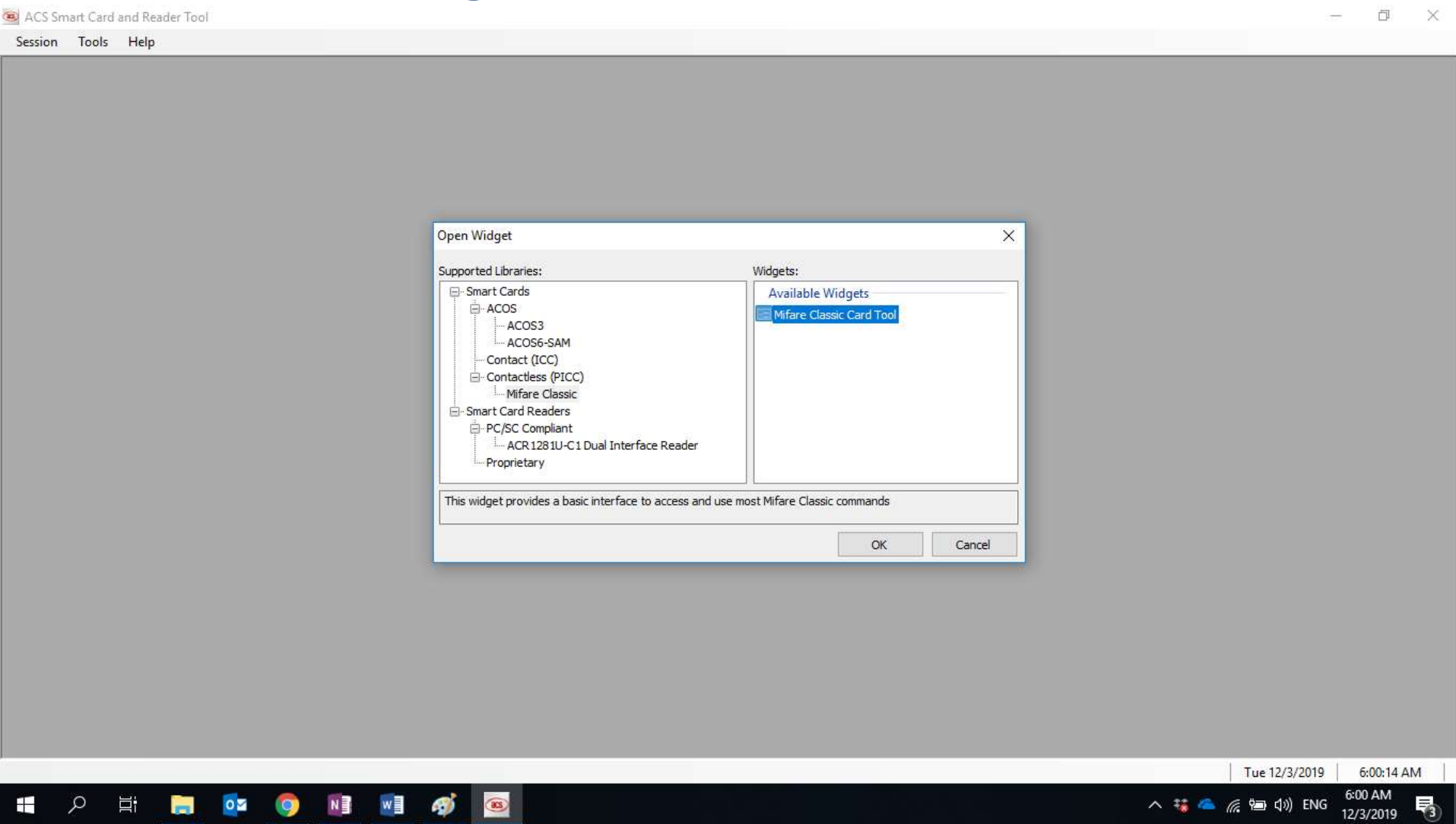
ACS Smart Card Reader Tool

- ☐ Enables the user to control the smart card reader from a Host PC.
- ☐ Provides means to send commands from the smart card reader to the smart card.
- ☐ There is a PCSC driver installed on the Host PC that allows communication with the Smart Card reader.
- ☐ The ACS Smart card and reader tool talks to the driver thorough standard APIs provided by the PCSC standard.

MiFARE Ultralight EV1:

- ❑ Contactless smart card with usage as single trip or limited use tickets in public transit, day passes for events, access token etc.
- ❑ It is just a memory card which can be seen as an alternative to a magnetic stripe card, but has additional functionalities.
- ❑ It has about 64 bytes of memory and has 48 bytes for storing data.
- ❑ You can read and write as many number of times to the MiFARE Ultralight EV1 card.
- ❑ It can also be password authenticated, but for our lab session, we use it as free-access cards.

Using ACS Smart Card and Reader Tool for MiFARE Ultralight Card



Establishing Connection with Smart Card Reader

ACS Smart Card and Reader Tool - [Mifare Classic Card Tool]

Session Tools Help

Mifare Classic Commands List

- Card Properties**
 - Card UID
- Card Security**
 - Load Key
 - Authenticate Block
- Data Block Commands**
 - Read Block
 - Write Block
- Value Block Commands**
 - Store Value
 - Read Value
 - Copy Value Block
 - Increment Value
 - Decrement Value

New Connection ▾ Disconnect Reset Clear Logs Export Logs

New PC/SC Connection

Change Connection Settings

Tue 12/3/2019 5:59:18 AM

Windows taskbar: Search, File Explorer, Edge, Chrome, Word, Paint, ACS Smart Card and Reader Tool

System tray: Network, Volume, ENG, 5:59 AM 12/3/2019, 3 notifications

Selecting the interface of the Smart Card Reader

ACS Smart Card and Reader Tool - [Mifare Classic Card Tool]

Session Tools Help

Mifare Classic Commands List

Card Properties

Card UID

Card Security

Load Key Authenticate Block

Data Block Commands

Read Block Write Block

Value Block Commands

Store Value Read Value

Copy Value Block

Increment Value Decrement Value

New Connection Disconnect Reset Clear Logs Export Logs

Connect to a PC/SC-compliant Contactless Smartcard Reader

Reader: ACS ACR1281 1S Dual Reader PICC 0
ACS ACR1281 1S Dual Reader ICC 0
ACS ACR1281 1S Dual Reader PICC 0
Connect: ACS ACR1281 1S Dual Reader SAM 0

Share Modes
☒ Shared
☐ Exclusive

Protocol
Mifare Classic cards uses
T=CL protocol

Connect Cancel

Tue 12/3/2019 6:01:01 AM

Windows taskbar icons: Start, Search, Task View, File Explorer, Microsoft Edge, Google Chrome, Notepad, Word, Paint, ACS Smart Card and Reader Tool.

System tray icons: Network, Volume, Power, Language (ENG), Date/Time (Tue 12/3/2019 6:01:01 AM), Notifications (3).

ANSWER TO RESET

ACS Smart Card and Reader Tool - [Mifare Classic Card Tool]

Session Tools Help

Mifare Classic Commands List

Card Properties

Card UID

Card Security

Load Key Authenticate Block

Data Block Commands

Read Block Write Block

Value Block Commands

Store Value Read Value

Copy Value Block

Increment Value Decrement Value

ACS ACR1281 1S Dual Reader PICC 0

New Connection Disconnect Reset Clear Logs Export Logs

Connected to ACS ACR1281 1S Dual Reader PICC 0

ATR: 3B 8F 80 01 80 4F 0C A0 00 00 03 06 03 00 03 00 00 00 68

← Answer to Reset (ATR)

Reader ACS ACR1281 1S Dual Reader PICC 0 Share Mode Shared Protocol T=CL

Tue 12/3/2019 6:01:24 AM

Windows taskbar: Windows Start, Search, Task View, File Explorer, Microsoft Edge, Google Chrome, Notepad, Word, Paint, ACS Smart Card and Reader Tool, System Tray (Network, Volume, Power, ENG, 6:01 AM 12/3/2019, 3 notifications)

ANSWER TO RESET

4.2.2. ATR format for ISO 14443 Part 3 PICCs.

Byte	Value (Hex)	Designation	Description
0	3B _H	Initial Header	
1	8N _H	T0	Higher nibble 8 means: no TA1, TB1, TC1 only TD1 is following. Lower nibble N is the number of historical bytes (HistByte 0 to HistByte N-1)
2	80 _H	TD1	Higher nibble 8 means: no TA2, TB2, TC2 only TD2 is following. Lower nibble 0 means T = 0
3	01 _H	TD2	Higher nibble 0 means no TA3, TB3, TC3, TD3 following. Lower nibble 1 means T = 1
4 To 3+N	80 _H	T1	Category indicator byte, 80 means A status indicator may be present in an optional COMPACT-TLV data object
	4F _H	Tk	Application identifier Presence Indicator
	0C _H		Length
	RID		Registered Application Provider Identifier (RID) # A0 00 00 03 06
	SS		Byte for standard
	C0... C1		Bytes for card name
	00 00 00 00 _H	RFU	RFU # 00 00 00 00
4+N	UU	TCK	Exclusive-oring of all the bytes T0 to Tk

Table 4: ISO 14443 Part 3 ATR Format

Answer to Reset is the first response on power up from the card.

It only tells the Smart card reader about the type of card...

It does not uniquely identify the card...

ATR is required for the card reader to know whom he is talking to...

ANSWER TO RESET

Example:

ATR for Mifare 1K = {3B 8F 80 01 80 4F 0C A0 00 00 03 06 03 00 01 00 00 00 00 6A}

ATR											
Initial Header	T0	TD1	TD2	T1	Tk	Length	RID	Standard	Card Name	RFU	TCK
3B _H	8F _H	80 _H	01 _H	80 _H	4F _H	0C _H	A0 00 00 03 06 _H	03 _H	00 _H 01 _H	00 00 00 00 _H	6A _H

Where:

Length (YY) = 0C
 RID = A0 00 00 03 06 (PC/SC Workgroup)
 Standard (SS) = 03 (ISO 14443A, Part 3)
 Card Name (C0 ... C1) =
 [00 01] (Mifare 1K)
 [00 02] (Mifare 4K)
 [00 03] (Mifare Ultralight)
 [00 26] (Mifare Mini)
 [FF 28] JCOP 30
 FF SAK undefined tags

From the ATR, try to identify what card you are working with...

Memory Map of Mifare Ultralight:

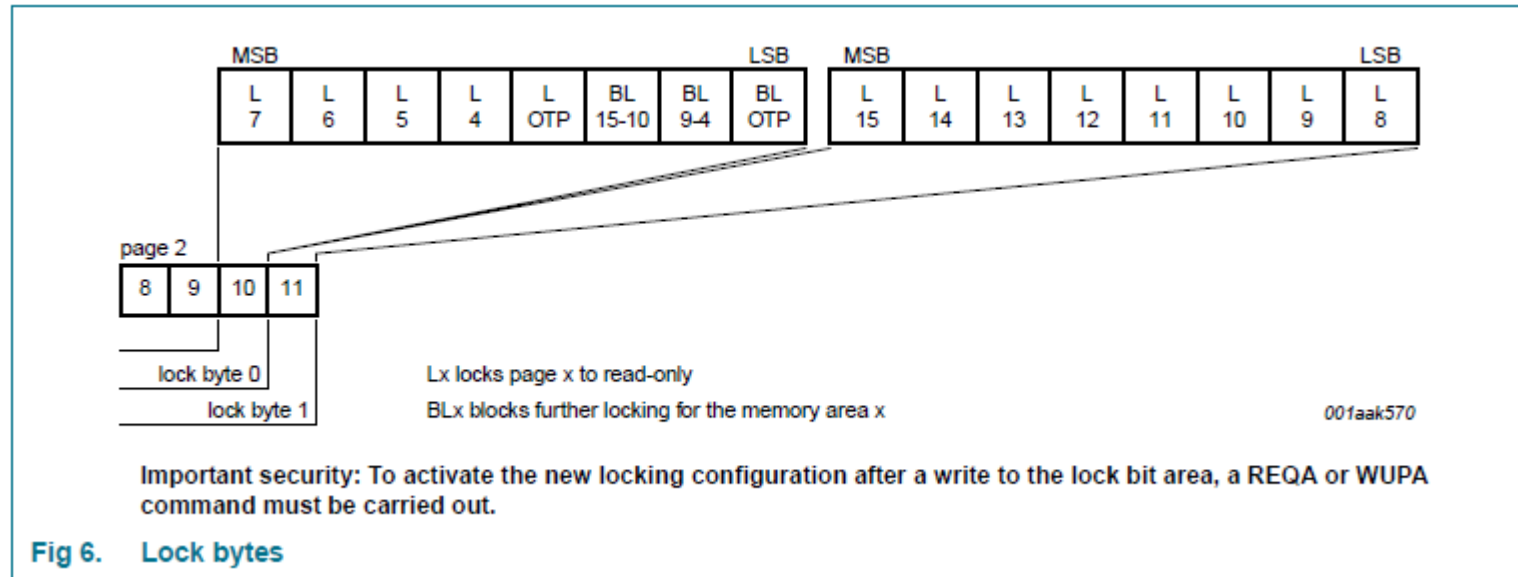
Byte Number	0	1	2	3	Page
Serial Number	SN0	SN1	SN2	BCC0	0
Serial Number	SN3	SN4	SN5	SN6	1
Internal / Lock	BCC1	Internal	Lock0	Lock1	2
OTP	OPT0	OPT1	OTP2	OTP3	3
Data read/write	Data0	Data1	Data2	Data3	4
Data read/write	Data4	Data5	Data6	Data7	5
Data read/write	Data8	Data9	Data10	Data11	6
Data read/write	Data12	Data13	Data14	Data15	7
Data read/write	Data16	Data17	Data18	Data19	8
Data read/write	Data20	Data21	Data22	Data23	9
Data read/write	Data24	Data25	Data26	Data27	10
Data read/write	Data28	Data29	Data30	Data31	11
Data read/write	Data32	Data33	Data34	Data35	12
Data read/write	Data36	Data37	Data38	Data39	13
Data read/write	Data40	Data41	Data42	Data43	14
Data read/write	Data44	Data45	Data46	Data47	15

512 bits
Or
64 bytes

In accordance with ISO/IEC 14443-3 Check Byte0 (BCC0) is defined as $CT \oplus SN0 \oplus SN1 \oplus SN2$ and Check Byte 1 (BCC1) is defined as $SN3 \oplus SN4 \oplus SN5 \oplus SN6$.

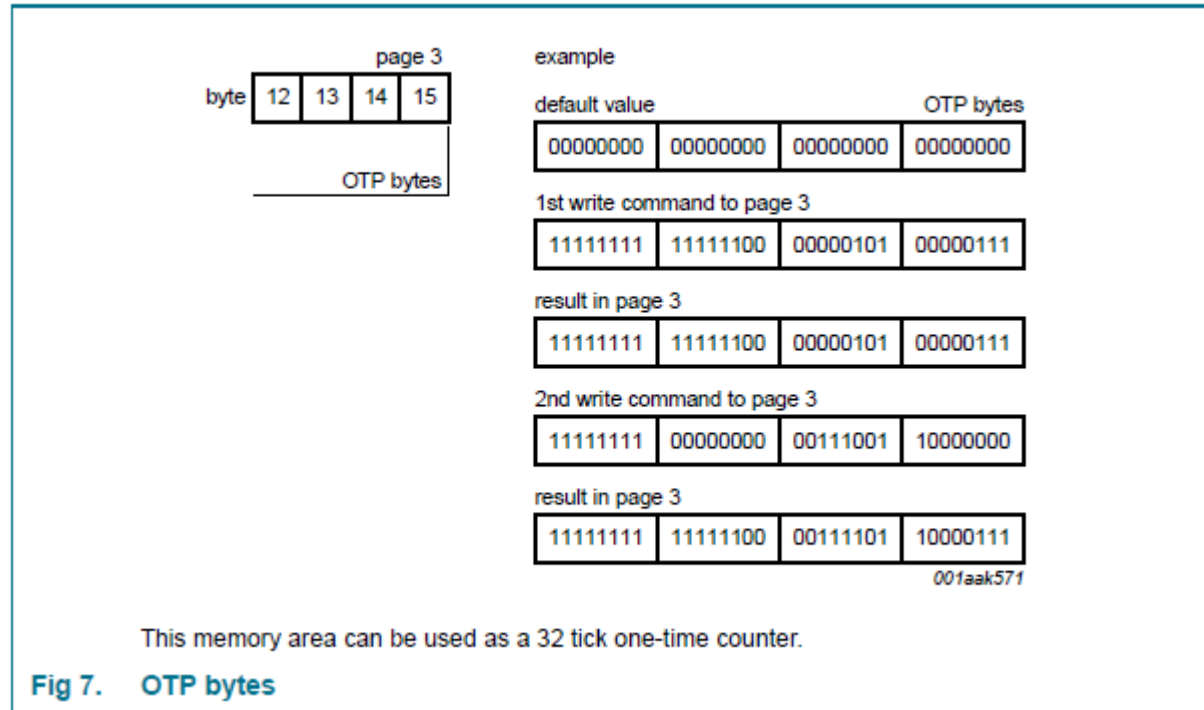
SN0 holds the Manufacturer ID for NXP Semiconductors (04h) in accordance with ISO/IEC 14443-3 and ISO/IEC 7816-6 AMD.1

Lock bits of Mifare Ultralight:



Can be used to individually lock pages in the memory.
Beware: These lock bits are 0 by default and once-programmed, the corresponding pages can never be written to.

OTP bits of Mifare Ultralight:



One-Time Programmable bytes in the memory.

Beware: These bits are 0 by default and once a bit is programmed to 1, it can never be reverted back to 0.

Memory Map of Mifare Ultralight:

Byte Number	0	1	2	3	Page
Serial Number	SN0	SN1	SN2	BCC0	0
Serial Number	SN3	SN4	SN5	SN6	1
Internal / Lock	BCC1	Internal	Lock0	Lock1	2
OTP	OPT0	OPT1	OTP2	OTP3	3
Data read/write	Data0	Data1	Data2	Data3	4
Data read/write	Data4	Data5	Data6	Data7	5
Data read/write	Data8	Data9	Data10	Data11	6
Data read/write	Data12	Data13	Data14	Data15	7
Data read/write	Data16	Data17	Data18	Data19	8
Data read/write	Data20	Data21	Data22	Data23	9
Data read/write	Data24	Data25	Data26	Data27	10
Data read/write	Data28	Data29	Data30	Data31	11
Data read/write	Data32	Data33	Data34	Data35	12
Data read/write	Data36	Data37	Data38	Data39	13
Data read/write	Data40	Data41	Data42	Data43	14
Data read/write	Data44	Data45	Data46	Data47	15

512 bits
Or
64 bytes

There are about 48 user programmable bytes.

They can be read and written any number of times and does not have any access control.

Table 8: Mifare Ultralight Memory Map

Read Memory:

ACS Smart Card and Reader Tool - [Mifare Classic Card Tool]

Session Tools Help

Mifare Classic Commands List

- Card Properties**
 - Card UID
- Card Security**
 - Load Key
 - Authenticate Block
- Data Block Commands**
 - Read Block
 - Write Block
- Value Block Commands**
 - Store Value
 - Read Value
 - Copy Value Block
 - Increment Value
 - Decrement Value

New Connection Disconnect Reset Clear Logs Export Logs

ACS ACR1281 1S Dual Reader PICC 0

Read Block

Block To Read: 00

Length : 10

Show in ASCII: ☐

Read Cancel

Input Page Number and length of bytes to read...

Reader ACS ACR1281 1S Dual Reader PICC 0 Share Mode Shared Protocol T=CL

Tue 12/3/2019 8:13:58 AM

8:13 AM 12/3/2019

Read Memory:

ACS Smart Card and Reader Tool - [Mifare Classic Card Tool]

Session Tools Help

Mifare Classic Commands List

Card Properties

Card UID

Card Security

Load Key Authenticate Block

Data Block Commands

Read Block Write Block

Value Block Commands

Store Value Read Value

Copy Value Block

Increment Value Decrement Value

ACS ACR1281 1S Dual Reader PICC 0

New Connection Disconnect Reset Clear Logs Export Logs

> Read Block
> FF B0 00 00 10
< 04 CD AB EA C2 37 30 80 45 48 00 00 00 00 00
< 90 00

← Bytes read from memory

Data: 04 CD AB EA C2 37 30 80 45 48 00 00 00 00 00

Try to Read the OTP bits and the Lock bits in your card...
Input the corresponding page number for retrieving that particular memory.

Reader ACS ACR1281 1S Dual Reader PICC 0 Share Mode Shared Protocol T=CL

Tue 12/3/2019 8:18:17 AM

8:18 AM 12/3/2019

Read Memory:

ACS Smart Card and Reader Tool - [Mifare Classic Card Tool]

Session Tools Help

Mifare Classic Commands List

Card Properties

Card UID

Card Security

Load Key Authenticate Block

Data Block Commands

Read Block Write Block

Value Block Commands

Store Value Read Value

Copy Value Block

Increment Value Decrement Value

ACS ACR1281 1S Dual Reader PICC 0

New Connection Disconnect Reset Clear Logs Export Logs

> Read Block
> FF B0 00 04 04
< 00 00 00 00
< 90 00

← Observe the command structure...

Data: 00 00 00 00

Command	Class	INS	P1	P2	Le
Read Binary Blocks	FF _H	B0 _H	00 _H	Block Number	Number of Bytes to Read

Reader ACS ACR1281 1S Dual Reader PICC 0 Share Mode Shared Protocol T=CL

Tue 12/3/2019 8:24:10 AM

8:24 AM 12/3/2019

Update Memory:

ACS Smart Card and Reader Tool - [Mifare Classic Card Tool]

Session Tools Help

Mifare Classic Commands List

- Card Properties
 - Card UID
- Card Security
 - Load Key
 - Authenticate Block
- Data Block Commands
 - Read Block
 - Write Block**
- Value Block Commands
 - Store Value
 - Read Value
 - Copy Value Block
 - Increment Value
 - Decrement Value

ACS ACR1281 1S Dual Reader PICC 0

New Connection Disconnect Reset Clear Logs Export Logs

> Read Block
> FF B0 00 04 04
< 00 00 00 00
< 90 00
Data: 00 00 00 00

Write Block

Target Block: 04 Length Of Data: 04

Data To Write

10	11	12	13												
----	----	----	----	--	--	--	--	--	--	--	--	--	--	--	--

Write Cancel

Reader ACS ACR1281 1S Dual Reader PICC 0 Share Mode Shared Protocol T=CL

Tue 12/3/2019 8:31:54 AM

Update Memory:

ACS Smart Card and Reader Tool - [Mifare Classic Card Tool]

Session Tools Help

Mifare Classic Commands List

Card Properties

Card UID

Card Security

Load Key Authenticate Block

Data Block Commands

Read Block Write Block

Value Block Commands

Store Value Read Value

Copy Value Block

Increment Value Decrement Value

ACS ACR1281 1S Dual Reader PICC 0

New Connection Disconnect Reset Clear Logs Export Logs

> Read Block
> FF B0 00 04 04
< 00 00 00 00
< 90 00
Data: 00 00 00 00

> Write Block
> FF D6 00 04 04 10 11 12 13
< 90 00
Write Block OK

← Response for Write Command...

Perform read to the same memory block to see if write was successful...

Reader ACS ACR1281 1S Dual Reader PICC 0 Share Mode Shared Protocol T=CL

Tue 12/3/2019 8:33:07 AM

8:33 AM 12/3/2019

Update Memory:

ACS Smart Card and Reader Tool - [Mifare Classic Card Tool]

Session Tools Help

Mifare Classic Commands List

Card Properties

Card UID

Card Security

Load Key Authenticate Block

Data Block Commands

Read Block Write Block

Value Block Commands

Store Value Read Value

Copy Value Block

Increment Value Decrement Value

ACS ACR1281 1S Dual Reader PICC 0

New Connection Disconnect Reset Clear Logs Export Logs

> Read Block
> FF B0 00 04 04
< 00 00 00 00
< 90 00
Data: 00 00 00 00

< Old Data

Try to observe the structure of the write command.

> Write Block
> FF D6 00 04 04 10 11 12 13
< 90 00
Write Block OK

> Read Block
> FF B0 00 04 04
< 10 11 12 13
< 90 00
Data: 10 11 12 13

< New Data

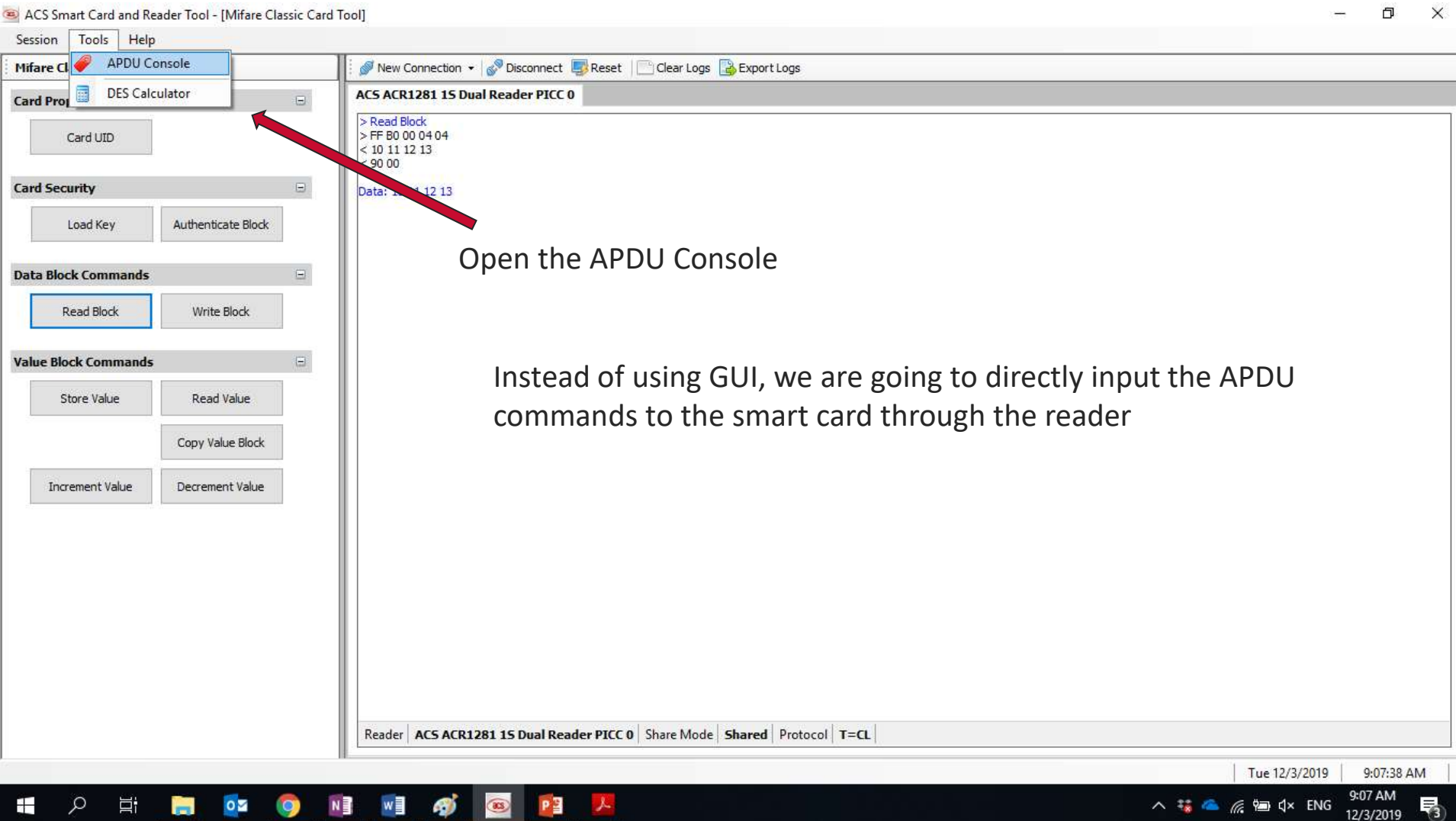
Command	Class	INS	P1	P2	Le	Data In
Update Binary Blocks	FF _H	D6 _H	00 _H	Block Number	Number of Bytes to Update	Block Data (Multiple of 16 Bytes)

Reader ACS ACR1281 1S Dual Reader PICC 0 Share Mode Shared Protocol T=CL

Tue 12/3/2019 8:34:48 AM

8:34 AM 12/3/2019

Sending Commands through APDU Console



The screenshot shows the 'ACS Smart Card and Reader Tool - [Mifare Classic Card Tool]' window. The 'Tools' menu is open, and the 'APDU Console' option is highlighted. A red arrow points from the text 'Open the APDU Console' to this menu item. The main window displays a connection to 'ACS ACR1281 1S Dual Reader PICC 0' and shows a 'Read Block' command being executed, with the resulting data '10 11 12 13' displayed. The left sidebar contains various tool categories: 'Card UID', 'Card Security', 'Data Block Commands' (with 'Read Block' selected), and 'Value Block Commands'. The bottom status bar shows 'Reader: ACS ACR1281 1S Dual Reader PICC 0', 'Share Mode: Shared', and 'Protocol: T=CL'.

Session Tools Help

Mifare Classic Card Tool

APDU Console

DES Calculator

Card UID

Card Security

Load Key Authenticate Block

Data Block Commands

Read Block Write Block

Value Block Commands

Store Value Read Value

Copy Value Block

Increment Value Decrement Value

ACS ACR1281 1S Dual Reader PICC 0

> Read Block
> FF B0 00 04 04
< 10 11 12 13
< 90 00
Data: 10 11 12 13

Open the APDU Console

Instead of using GUI, we are going to directly input the APDU commands to the smart card through the reader

Reader ACS ACR1281 1S Dual Reader PICC 0 Share Mode Shared Protocol T=CL

Tue 12/3/2019 9:07:38 AM

9:07 AM 12/3/2019

Sending Commands through APDU Console

ACS Smart Card and Reader Tool - [Mifare Classic Card Tool]

Session Tools Help

Mifare Classic Commands List

Card Properties

Card UID

Card Security

Load Key Authenticate Block

Data Block Commands

Read Block Write Block

Value Block Commands

Store Value Read Value

Copy Value Block

Increment Value Decrement Value

ACS ACR1281 1S Dual Reader PICC 0

New Connection Disconnect Reset Clear Logs Export Logs

> Read Block
> FF B0 00 04 04
< 10 11 12 13
< 90 00

Data: 10 11 12 13

> APDU
> FF B0 00 04 04
< 10 11 12 13
< 90 00

Get Response here!!!

APDU Command Console

Active Reader: **ACS ACR1281 1S Dual Reader PICC 0**

APDU Command

CLA	INS	P1	P2	Lc	Le
FF	B0	00	04	00	04

Input Command here!!!

Data In Parameters

	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F
► +00																
+10																
+20																

Data Out Result

	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F
► +00	10	11	12	13												
+10																
+20																

SW1 SW2
90 00

Transmit Clear Close

0 - The operation completed successfully

Reader **ACS ACR1281 1S Dual Reader PICC 0** Share Mode **Shared** Protocol **T=CL**

Tue 12/3/2019 9:12:38 AM

Windows Taskbar: File Explorer, Edge, Word, PowerPoint, PDF Reader, ACS Tool, etc.

AGENDA:

- ❑ Hands-on session with Smart Cards and understanding how to communicate with Smart Cards.
- ❑ We will introduce the students to the hardware and software components in a basic smart card setup and trying to familiarize with them.
- ❑ We will work with two types of Smart cards:
 - ❑ Mifare Ultralight Memory Card
 - ❑ CEPAS 2.0 card (Your ID card/EZLink Card)
 - ❑ Mifare Classic 1K

CEPAS 2.0 Card

- ☐ Specification for **C**ontactless **E**-**P**urse **A**pplication
- ☐ Singaporean specification for electronic money card
- ☐ Singaporean vision of single smart card specification for all services.
- ☐ Provides details on the following:
 - ☐ Commands to communicate with the card.
 - ☐ How the data is managed inside the card.
 - ☐ Crypto Application to secure data in the card (Debit and Credit functions)
- ☐ NETS and EZ-Link cards have to adhere to CEPAS specifications in Singapore.

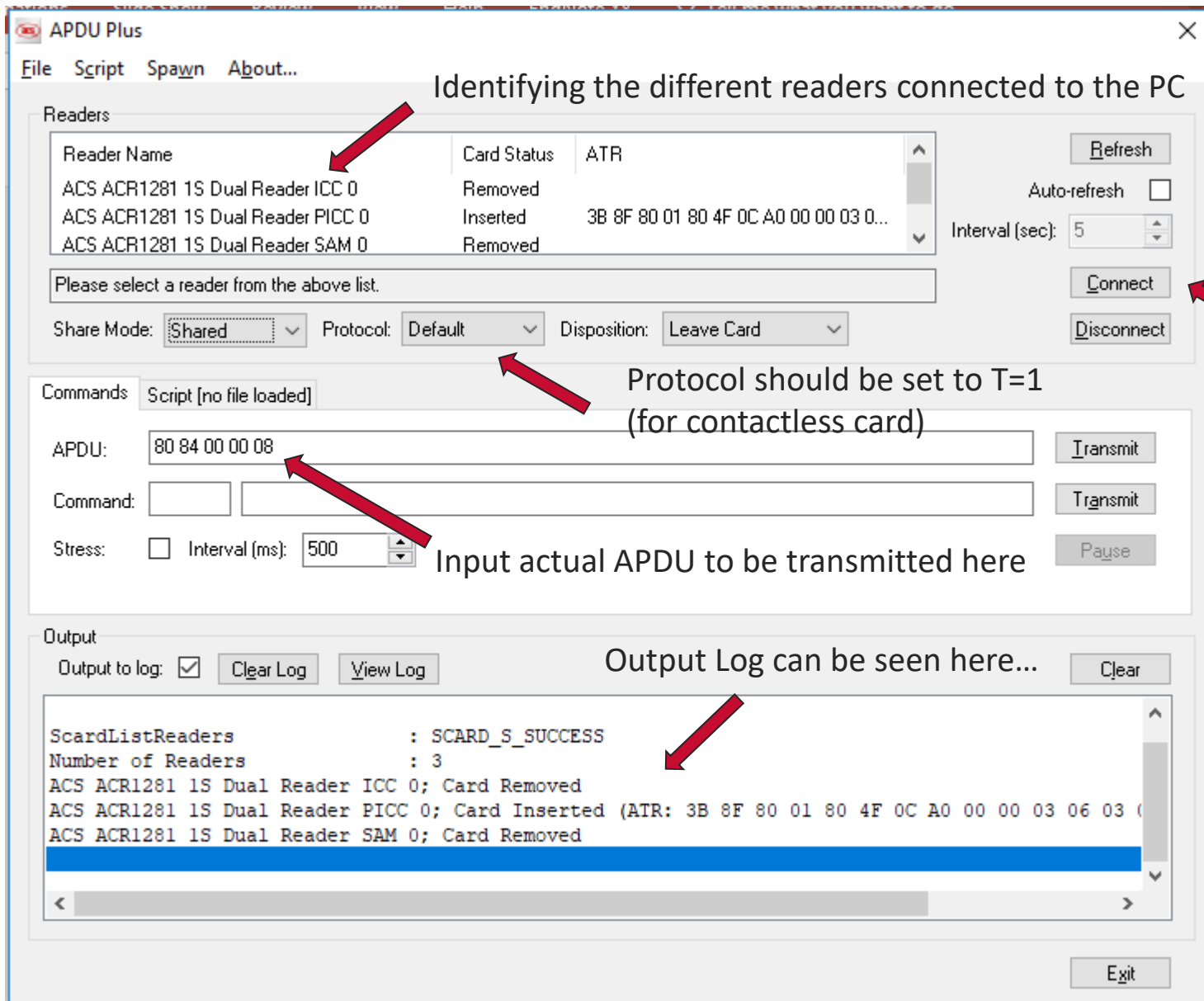
CEPAS 2.0 Card

- ❑ Credit and Debit functions are protected by cryptographic algorithms. Hence, we cannot tamper with that.
- ❑ We can read purse balance from the purse file and also read transaction log.
- ❑ Specific commands can be provided to the smart card based on the CEPAS specifications from the APDU Plus tool to evoke proper responses.

APDU Plus

- ❑ Application from Advanced Card Systems which is used to send plain APDU commands to the smart card through the smart card reader.
- ❑ It is not customized for any specific smart card and hence the specific APDU command has to be input to the tool to communicate with the smart card.
- ❑ We will use this tool to read purse balance and transaction log from a CEPAS 2.0 card.

APDU Plus



READING PURSE BALANCE OF CEPAS CARD

- ☐ Purse balance is stored in a Purse File.
- ☐ Input the APDU “90 32 03 00 00” to read purse balance.

CLA	'90' (proprietary class)
INS	'32'
P1	Encodes a short EF identifier of the File ID of the Purse File - Pf
P2	RFU
L_c field	'00' (no authentication), '01' (Read Transaction Log) or '0A' (with authentication)
Data field	See Read Purse Command Data
L_e field	Number of byte to be read

- ☐ Each purse file is identified by unique ID. Our ID is??
- ☐ There can be multiple Purse Files. Why???
- ☐ The above command is authenticated or not??
- ☐ Purse balance is specified by 3 byte signed integer. Why signed??
- ☐ Error Correction to preserve integrity of purse balance.

STRUCTURE OF PURSE BALANCE RECORD

Data object	Length	Security
CEPAS version	1 byte	None
Purse status	1 byte	None
Purse balance	3 bytes	None
AutoLoad amount	3 bytes	None
CAN	8 bytes	None
CSN	8 bytes	None
Purse expiry date (Julian Date)	2 bytes	None
Purse creation date (Julian Date)	2 bytes	None
Last credit transaction TRP	4 bytes	None
Last credit transaction header	8 bytes	None
No. of records in transaction logfile	1 byte	None
Issuer-specific data length	1 byte	None
Last transaction TRP	4 bytes	None
Last transaction record	16 bytes	None
Issuer-specific data	X bytes	None

READ TRANSACTION LOG OF CEPAS CARD

- ☐ Transaction log records are also sometimes stored along with Purse File in the card.
- ☐ Best Data types to store Transaction log. FIFO or LIFO??
- ☐ When is the transaction log updated?
- ☐ Input the APDU “90 32 03 00 01 00 00” to read transaction log record.
- ☐ In this case, is the transaction log record stored in the same file as the Purse file??

Transaction log record format			
Transaction header			Transaction user data
Transaction type	Transaction amount	Transaction DateTime	
1 byte	3 bytes	4 bytes	8 bytes

ACTIVITY:

- ☐ Read the Purse Balance and Transaction log of your card using an Android App for reading Smart Card Readers (SingCARD and many more are available...)
- ☐ Note it down and pass the card to your neighbouring group.
- ☐ Manually read the purse balance and transaction record of the shared card and check with the group if the information is correct.

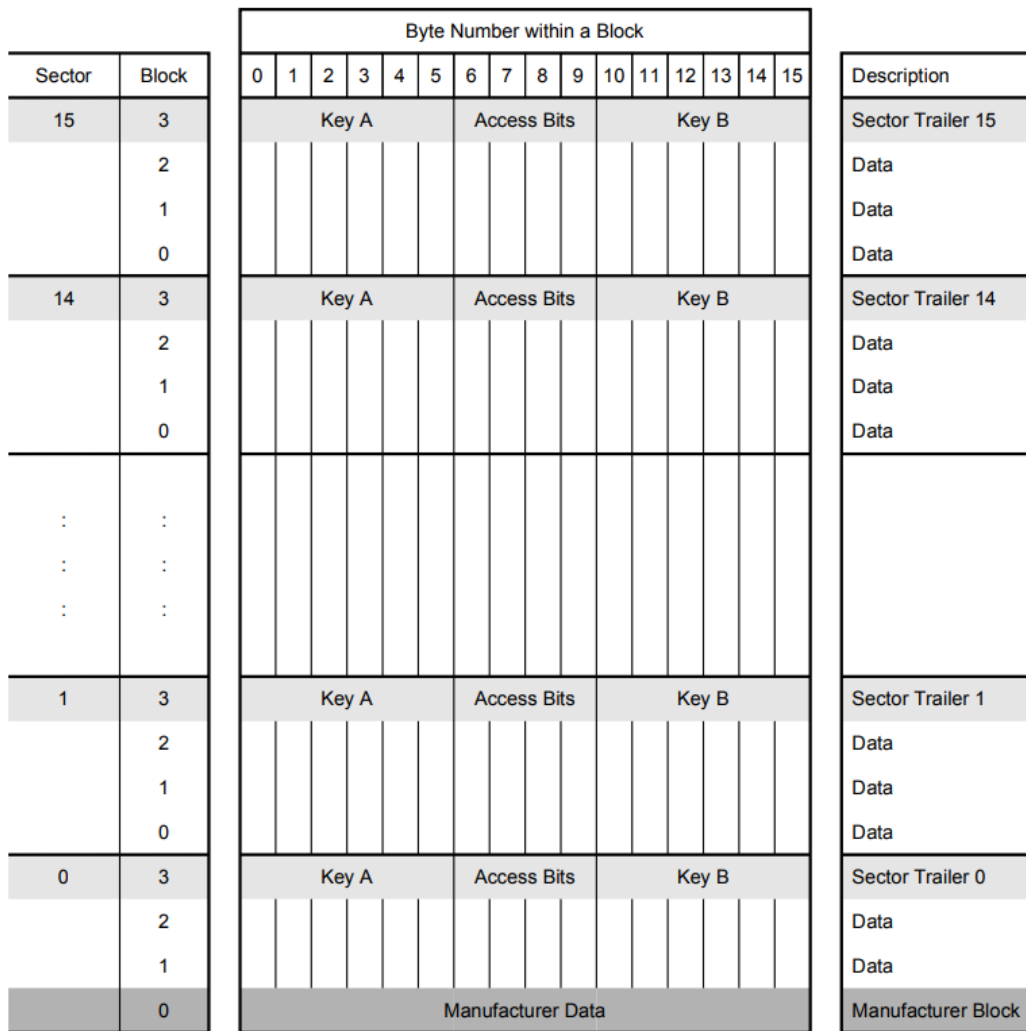
MIFARE CLASSIC:

- ❑ Memory card like MIFARE Ultralight, but with additional memory and additional functionalities.
- ❑ Uses: Public transport ticketing and as access tokens.
- ❑ 1Kilobyte of memory (organized in 16 sectors each with four blocks).
- ❑ MIFARE Ultralight:
 - ❑ 64 bytes (16 blocks with four bytes)
- ❑ All memory blocks are **authenticated** before they are accessed.

MIFARE CLASSIC:

- ❑ In addition to read and write, you can also do basic arithmetic on data (add and sub).

MEMORY MAP OF MIFARE CLASSIC:



001aan011

Figure 5. Memory organization

- ☐ Each sector has four blocks
- ☐ First three blocks within a sector are data blocks.
- ☐ Fourth block in a sector is the trailer block.
- ☐ Trailer block stores two keys and the access rights of the block.

DATA BLOCK:

- ☐ Each Sector contains three data blocks
- ☐ You can read and write any data to the data blocks
- ☐ How do you read/write data block??
 - ☐ You need to authenticate yourself...
 - ☐ How do you authenticate yourself?? Using secret keys

READ AND WRITE BLOCK:

- ☐ First, load the key you want to use for authentication in the reader.
- ☐ Choose the block 8 and try to authenticate using the default key “0X FF FF FF FF FF FF”.
- ☐ If you authenticate a single block, you authenticate the whole sector.
- ☐ Authenticate block 8, you can read/write blocks 8,9,10 and 11 (All belonging to Sector 3)

Load Authentication keys on the card reader

ACS Smart Card and Reader Tool - [Mifare Classic Card Tool]

Session Tools Help

Mifare Classic Commands List

Card Properties

Card UID

Card Security

Load Key Authenticate Block

Data Block Commands

Read Block Write Block

Value Block Commands

Store Value Read Value

Copy Value Block

Increment Value Decrement Value

ACS ACR1281 1S Dual Reader PICC 0

New Connection Disconnect Reset Clear Logs Export Logs

Load Key

Key Structure: 20

Key Number: 00

Key To Load

10	11	12	13	14	15
----	----	----	----	----	----

☐ Use default sector keys

Load Cancel

00 – For volatile (erasable key)
20 – For non-volatile (non erasable key)

Key number on card (Max 32)

Input all values in hex

Reader ACS ACR1281 1S Dual Reader PICC 0 Share Mode Shared Protocol T=CL

Tue 12/3/2019 6:10:26 AM

6:10 AM 12/3/2019

Load Authentication keys on the card reader

Command

Command	Class	INS	P1	P2	Le	Data In
Load Authentication Keys	FF _H	82 _H	Key Structure	Key Number	06 _H	Key

- ☐ Can store upto a maximum of 32 keys.
- ☐ Each key can be programmed to be either volatile or non-volatile.

Load Authentication keys on the card reader

ACS Smart Card and Reader Tool - [Mifare Classic Card Tool]

Session Tools Help

Mifare Classic Commands List

Card Properties

Card UID

Card Security

Load Key Authenticate Block

Data Block Commands

Read Block Write Block

Value Block Commands

Store Value Read Value

Copy Value Block

Increment Value Decrement Value

ACS ACR1281 1S Dual Reader PICC 0

New Connection Disconnect Reset Clear Logs Export Logs

> Load Key
> FF 82 20 00 06 10 11 12 13 14 15
< 90 00

Load Key OK

90 00 is the code for success of a command response!!!

Reader ACS ACR1281 1S Dual Reader PICC 0 Share Mode Shared Protocol T=CL

Tue 12/3/2019 6:54:41 AM

6:54 AM 12/3/2019

DATA BLOCKS:

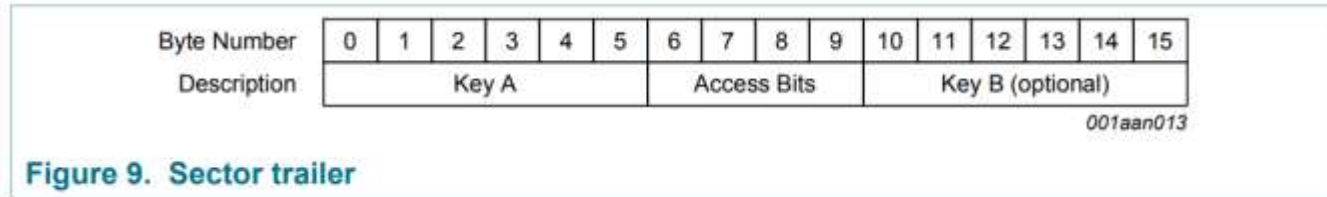
- ☐ The first three blocks in a sector are the data blocks.
- ☐ The data block can act as either **read/write** block or **value** block.
- ☐ If you store a 16 byte random value in a block, it becomes a read/write block.
- ☐ If you store 4 byte data in a specific format, it becomes a **value** block.

Byte Number	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
Description	value				$\overline{\text{value}}$				value				adr	$\overline{\text{adr}}$	adr	$\overline{\text{adr}}$

001aan018

- ☐ Why store data repeatedly??

TRAILER BLOCK:



- ❑ Contains **keys** (Key A and Key B) for the particular sector
- ❑ Contains **access information** for each data block and trailer block.
- ❑ Bytes 6,7 and 8 determine access permissions for data and trailer block.
- ❑ What is the format of access bits??

TRAILER BLOCK:

- ❑ Access rights are encoded in three bits for each block.

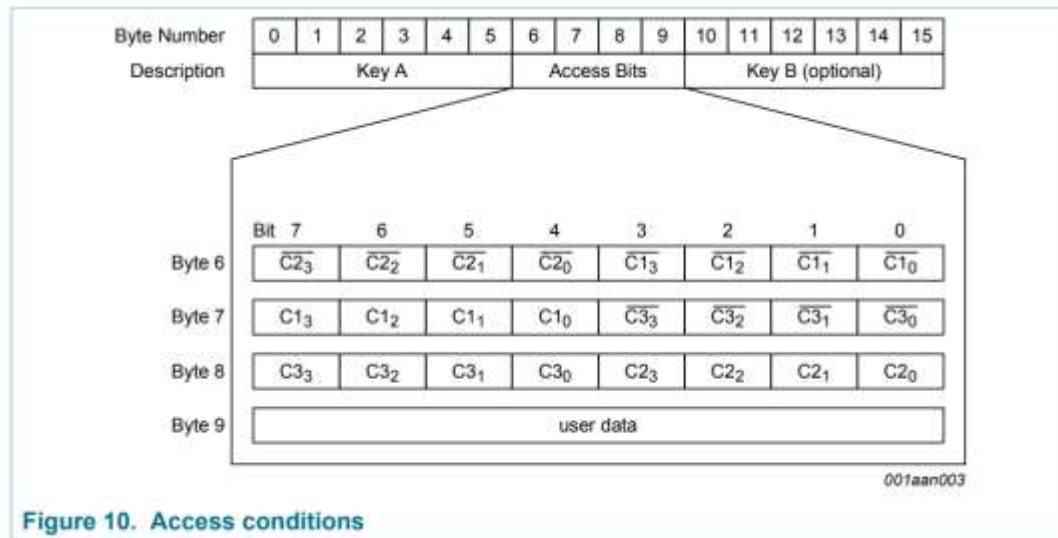


Figure 10. Access conditions

Access Bits	Valid Commands		Block	Description
$C1_3, C2_3, C3_3$	read, write	→	3	sector trailer
$C1_2, C2_2, C3_2$	read, write, increment, decrement, transfer, restore	→	2	data block
$C1_1, C2_1, C3_1$	read, write, increment, decrement, transfer, restore	→	1	data block
$C1_0, C2_0, C3_0$	read, write, increment, decrement, transfer, restore	→	0	data block

TRAILER BLOCK ACCESS BITS:

- ❑ Access rights are encoded in three bits for each block.

Table 7. Access conditions for the sector trailer

Access bits			Access condition for						Remark
			KEYA		Access bits		KEYB		
C1	C2	C3	read	write	read	write	read	write	
0	0	0	never	key A	key A	never	key A	key A	Key B may be read ^[1]
0	1	0	never	never	key A	never	key A	never	Key B may be read ^[1]
1	0	0	never	key B	key A B	never	never	key B	
1	1	0	never	never	key A B	never	never	never	
0	0	1	never	key A	key A	key A	key A	key A	Key B may be read, transport configuration ^[1]
0	1	1	never	key B	key A B	key B	never	key B	
1	0	1	never	never	key A B	key B	never	never	
1	1	1	never	never	key A B	never	never	never	

[1] For this access condition key B is readable and may be used for data

DATA BLOCK ACCESS BITS:

- Access rights are encoded in three bits for each block.

Table 8. Access conditions for data blocks

Access bits			Access condition for				Application
C1	C2	C3	read	write	increment	decrement, transfer, restore	
0	0	0	key A B	key A B	key A B	key A B	transport configuration ^[1]
0	1	0	key A B	never	never	never	read/write block ^[1]
1	0	0	key A B	key B	never	never	read/write block ^[1]
1	1	0	key A B	key B	key B	key A B	value block ^[1]
0	0	1	key A B	never	never	key A B	value block ^[1]
0	1	1	key B	key B	never	never	read/write block ^[1]
1	0	1	key B	never	never	never	read/write block ^[1]
1	1	1	never	never	never	never	read/write block

ACTIVITY:

- ☐ Calculate Access Bits for Following Condition:
 - ☐ **Data Block 0:** Read by Key A/B, Cannot be written. Cannot be incremented. But, can be decremented using key A/B
 - ☐ **Data Block 1:** Read by Key A/B, Cannot be written, incremented or decremented
 - ☐ **Data Block 2:** Read and written only by Key B, cannot be incremented or decremented
 - ☐ **Key A:** Never Read, Write using Key B
 - ☐ **Key B:** Never Read, Write using Key B
 - ☐ **Access Bits:** Read using Key A/B, Write using Key B

VALUE BLOCK:

Byte Number	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
Description	value				$\overline{\text{value}}$				value				adr	$\overline{\text{adr}}$	adr	$\overline{\text{adr}}$

001aan018

Table 4. Value block format example

Byte Number	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
Description	value				$\overline{\text{value}}$				value				adr	$\overline{\text{adr}}$	adr	$\overline{\text{adr}}$
Values [hex]	87	D6	12	00	78	29	ED	FF	87	D6	12	00	11	EE	11	EE

- ☐ You can try to write any value to a data block in the above specified format.
- ☐ Subsequently, you can try to perform basic increment, decrement operations on this value.

DATA BLOCKS:

- ❑ You can only read/write data to a read/write block.
- ❑ But, you can perform basic arithmetic (addition and subtraction) over value blocks.

Byte Number	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
Description	value				$\overline{\text{value}}$				value				adr	$\overline{\text{adr}}$	adr	$\overline{\text{adr}}$

001aan018

THREE PASS AUTHENTICATION SEQUENCE:



I want to read sector 3 using Key A

Wait...let me check if you can do that... (Checks access condition of sector 3).

Well, you need to send me a response to this challenge C1....

Wait...let me send the response for you... (Response calculation with challenge C and key input K)

Here you go... This is my challenge C2 and my response R1 to your challenge C1...

Let me check if your response R1 is correct...

It is correct... Ah you are my long time friend Alex!!
:P

Cmon man....You gotta trust me... ☺ Anyways, let me get my response for your challenge...
Here you go...My response R2 to your challenge C2.

No matter how many times we meet each other, I've got to do this every time to you man :P ... Response R2 Verified...

I wish I could ever get in contact with you my contactless friend :P

A Practical Attack on the MIFARE Classic

Gerhard de Koning Gans, Jaap-Henk Hoepman,
and Flavio D. Garcia

Institute for Computing and Information Sciences
Radboud University Nijmegen
P.O. Box 9010, 6500 GL Nijmegen, The Netherlands
`gkoningg@sci.ru.nl`,
`jhh@cs.ru.nl`,
`flaviog@cs.ru.nl`

Abstract. The MIFARE Classic is the most widely used contactless smart card in the market. Its design and implementation details are kept secret by its manufacturer. This paper studies the architecture of the card and the communication protocol between card and reader. Then it gives a practical, low-cost, attack that recovers secret information from the memory of the card. Due to a weakness in the pseudo-random generator, we are able to recover the keystream generated by the CRYPTO1 stream cipher. We exploit the malleability of the stream cipher to read *all* memory blocks of the first sector of the card. Moreover, we are able to read *any* sector of the memory of the card, provided that we know *one* memory block within this sector. Finally, and perhaps more damaging, the same holds for *modifying* memory blocks.

Activity: Use MIFARE Classic as an MRT Card

- **Basic Functionality:**
 - Top Up (Secure)
 - Debit Amount when tapping (Secure)
 - Read Purse Balance (Non-Secure)
 - Read Transaction Log (Non-Secure)
- **How do you realize these functionalities with the available features??**
- **Please attempt to write a pseudo code for each function:**

Activity: Use MIFARE Classic as an MRT Card

- **Basic Functionality:**
 - Top Up (Secure)
 - Debit Amount when tapping (Secure)
 - Read Purse Balance
 - Read Transaction Log
- **Please attempt to write a pseudo code for each function:**
 - You can use the features of value block, data blocks, access bits, keys etc.